

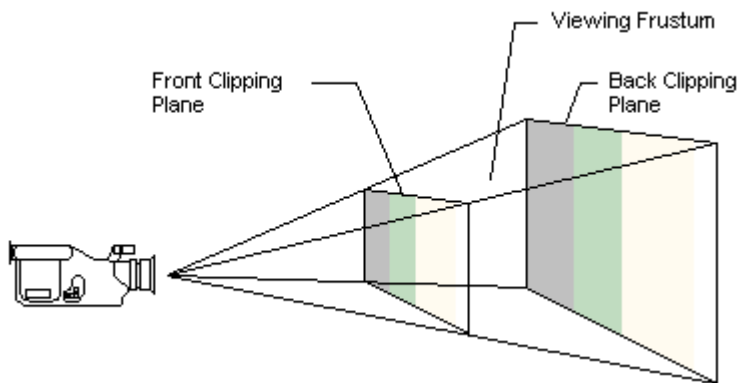
Viewports and clipping

Article • 10/20/2022

A *viewport* is a two-dimensional (2D) rectangle into which a 3D scene is projected. In Direct3D, the rectangle exists as coordinates within a Direct3D surface that the system uses as a rendering target. The projection transformation converts vertices into the coordinate system used for the viewport. A viewport is also used to specify the range of depth values on a render-target surface into which a scene will be rendered (usually 0.0 to 1.0).

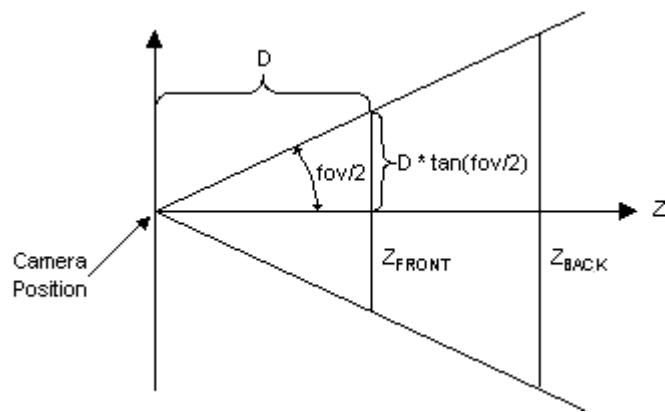
The Viewing Frustum

A viewing frustum is 3D volume in a scene positioned relative to the viewport's camera. The shape of the volume affects how models are projected from camera space onto the screen. The most common type of projection, a perspective projection, is responsible for making objects near the camera appear bigger than objects in the distance. For perspective viewing, the viewing frustum can be visualized as a pyramid, with the camera positioned at the tip as shown in the following illustration. This pyramid is intersected by a front and back clipping plane. The volume within the pyramid between the front and back clipping planes is the viewing frustum. Objects are visible only when they are in this volume.



If you imagine that you are standing in a dark room and looking through a square window, you are visualizing a viewing frustum. In this analogy, the near clipping plane is the window, and the back clipping plane is whatever finally interrupts your view - the skyscraper across the street, the mountains in the distance, or nothing at all. You can see everything inside the truncated pyramid that starts at the window and ends with whatever interrupts your view, and you can see nothing else.

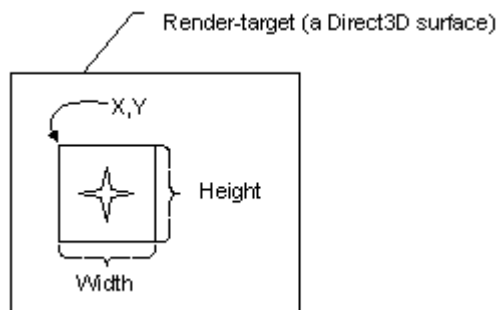
The viewing frustum is defined by fov (field of view) and by the distances of the front and back clipping planes, specified in z-coordinates, as shown in the following diagram.



In this diagram, the variable D is the distance from the camera to the origin of the space that was defined in the last part of the geometry pipeline - the viewing transformation. This is the space around which you arrange the limits of your viewing frustum. For information about how this D variable is used to build the projection matrix, see the [Projection transform](#)

Viewport Rectangle

A viewport struct contains four members (X , Y , $Width$, $Height$) that define the area of the render-target surface into which a scene will be rendered. These values correspond to the destination rectangle, or viewport rectangle, as shown in the following diagram.



The values you specify for the X , Y , $Width$, $Height$ members are screen coordinates relative to the upper-left corner of the render-target surface. The structure defines two additional members ($MinZ$ and $MaxZ$) that indicate the depth-ranges into which the scene will be rendered.

Direct3D assumes that the viewport clipping volume ranges from -1.0 to 1.0 in X , and from 1.0 to -1.0 in Y . These were the settings used most often by applications in the past. You can adjust the viewport aspect ratio before clipping using the [projection transform](#).

Note $MinZ$ and $MaxZ$ indicate the depth-ranges into which the scene will be rendered and are not used for clipping. Most applications will set these values to 0.0 and 1.0 , to enable the system to render to the entire range of depth values in the depth buffer. In some cases, you can achieve special effects by using other depth ranges. For instance, to

render a heads-up display in a game, you can set both values to 0.0 to force the system to render objects in a scene in the foreground, or you might set them both to 1.0 to render an object that should always be in the background.

The dimensions used in the X, Y, Width, Height members of a viewport struct define the location and dimensions of the viewport on the render-target surface. These values are in screen coordinates, relative to the upper-left corner of the surface.

Direct3D uses the viewport location and dimensions to scale the vertices to fit a rendered scene into the appropriate location on the target surface. Internally, Direct3D inserts these values into the following matrix that is applied to each vertex.

$$\begin{bmatrix} dwWidth/2 & 0 & 0 & 0 \\ 0 & -dwHeight/2 & 0 & 0 \\ 0 & 0 & dvMaxZ - dvMinZ & 0 \\ dwX + dwWidth/2 & dwHeight/2 + dwY & dvMinz & 1 \end{bmatrix}$$

This matrix scales vertices according to the viewport dimensions and desired depth range and translates them to the appropriate location on the render-target surface. The matrix also flips the y-coordinate to reflect a screen origin at the top-left corner with y increasing downward. After this matrix is applied, vertices are still homogeneous - that is, they still exist as [x,y,z,w] vertices - and they must be converted to non-homogeneous coordinates before being sent to the rasterizer.

Note Applications typically set MinZ and MaxZ to 0.0 and 1.0 respectively to cause the system to render to the entire depth range. However, you can use other values to achieve certain affects. For example, you might set both values to 0.0 to force all objects into the foreground, or set both to 1.0 to render all objects into the background.

Clearing a Viewport

Clearing the viewport resets the contents of the viewport rectangle on the render-target surface. It can also clear the rectangle in the depth and stencil buffer surfaces.

Set Up the Viewport for Clipping

The results of the projection matrix determine the clipping volume in projection space as:

$$-w_c \leq x_c \leq w_c$$

$$-w_c \leq y_c \leq w_c$$

$$0 \leq z_c \leq w_c$$

Where: x , y , z , and w represent the vertex coordinates after the projection transformation is applied. Any vertices that have an x -, y -, or z -component outside these ranges are clipped, if clipping is enabled (the default behavior).

Related topics

[Coordinate systems and geometry](#)

Vertex and index buffers

Article • 10/20/2022

Vertex buffers are memory buffers that contain vertex data; vertices in a vertex buffer are processed to perform transformation, lighting, and clipping. *Index buffers* are memory buffers that contain index data, which are integer offsets into vertex buffers, used to render primitives.

Vertex buffers can contain any vertex type - transformed or untransformed, lit or unlit - that can be rendered. You can process the vertices in a vertex buffer to perform operations such as transformation, lighting, or generating clipping flags. Transformation is always performed.

The flexibility of vertex buffers make them ideal staging points for reusing transformed geometry. You could create a single vertex buffer, transform, light, and clip the vertices in it, and render the model in the scene as many times as needed without re-transforming it, even with interleaved render state changes. This is useful when rendering models that use multiple textures: the geometry is transformed only once, and then portions of it can be rendered as needed, interleaved with the required texture changes. Render state changes made after vertices are processed take effect the next time the vertices are processed.

In this section

Topic	Description
Introduction to buffers	A buffer resource is a collection of fully typed data, grouped into elements. Buffers store data, such as texture coordinates in a <i>vertex buffer</i> , indexes in an <i>index buffer</i> , shader constants data in a <i>constant buffer</i> , position vectors, normal vectors, or device state.
Index buffers	<i>Index buffers</i> are memory buffers that contain index data, which are integer offsets into vertex buffers, used to render primitives.

Related topics

[Direct3D Graphics Learning Guide](#)

Introduction to buffers

Article • 10/20/2022

A buffer resource is a collection of fully typed data, grouped into elements. Buffers store data, such as texture coordinates in a *vertex buffer*, indexes in an *index buffer*, shader constants data in a *constant buffer*, position vectors, normal vectors, or device state.

A buffer element is made up of 1 to 4 components. Buffer elements can include packed data values (like R8G8B8A8 surface values), single 8-bit integers, or four 32-bit floating point values.

A buffer is created as an unstructured resource. Because it is unstructured, a buffer cannot contain any mipmap levels, it cannot get filtered when read, and it cannot be multisampled.

Buffer Types

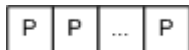
The following are the buffer resource types supported by Direct3D 11.

- [Vertex Buffer](#)
- [Index Buffer](#)
- [Constant Buffer](#)

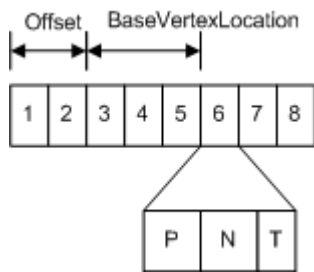
Vertex Buffer

A vertex buffer contains the vertex data used to define your geometry. Vertex data includes position coordinates, color data, texture coordinate data, normal data, and so on.

The simplest example of a vertex buffer is one that only contains position data. It can be visualized like the following illustration.



More often, a vertex buffer contains all the data needed to fully specify 3D vertices. An example of this could be a vertex buffer that contains per-vertex position, normal and texture coordinates. This data is usually organized as sets of per-vertex elements, as shown in the following illustration.



This vertex buffer contains per-vertex data; each vertex stores three elements (position, normal, and texture coordinates). The position and normal are each typically specified using three 32-bit floats and the texture coordinates using two 32-bit floats.

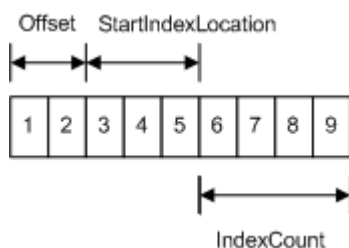
To access data from a vertex buffer you need to know which vertex to access, plus the following additional buffer parameters:

- Offset - the number of bytes from the start of the buffer to the data for the first vertex.
- BaseVertexLocation - the number of bytes from the offset to the first vertex used by the appropriate draw call.

Before you create a vertex buffer, you need to define its layout. After the input-layout object is created, you bind it to the [Input Assembler \(IA\) stage](#).

Index Buffer

Index buffers contain integer offsets into vertex buffers and are used to render primitives more efficiently. An index buffer contains a sequential set of 16-bit or 32-bit indices; each index is used to identify a vertex in a vertex buffer. An index buffer can be visualized like the following illustration.



The sequential indices stored in an index buffer are located with the following parameters:

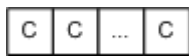
- Offset - the number of bytes from the base address of the index buffer.
- StartIndexLocation - specifies the first index buffer element from the base address and the offset. The start location represents the first index to render.
- IndexCount - the number of indices to render.

Start of Index Buffer = Index Buffer Base Address + Offset (bytes) + StartIndexLocation * ElementSize (bytes);

In this calculation, ElementSize is the size of each index buffer element, which is either two or four bytes.

Constant Buffer

A constant buffer allows you to efficiently supply shader constants data to the pipeline. You can use a constant buffer to store the results of the stream-output stage. Conceptually, a constant buffer looks just like a single-element vertex buffer, as shown in the following illustration.



Each element stores a 1-to-4 component constant, determined by the format of the data stored.

A constant buffer can only use a single bind flag , which cannot be combined with any other bind flag.

To read a shader-constant buffer from a shader, use an HLSL load function. Each shader stage allows up to 15 shader-constant buffers; each buffer can hold up to 4096 constants.

Related topics

[Vertex and index buffers](#)

Index buffers

Article • 10/20/2022

Index buffers are memory buffers that contain index data, which are integer offsets into vertex buffers, used to render primitives.

Index buffers are memory buffers that contain index data. Index data, or indices, are integer offsets into vertex buffers and are used to render primitives.

A vertex buffer contains vertices; therefore, you can draw a vertex buffer either with or without indexed primitives. However, because an index buffer contains indices, you cannot use an index buffer without a corresponding vertex buffer.

Index Buffer Description

An index buffer is described in terms of its capabilities, such as where it exists in memory, whether it supports reading and writing, and the type and number of indices it can contain.

Index buffer descriptions tell your application how an existing buffer was created. You provide an empty description structure for the system to fill with the capabilities of a previously created index buffer.

Index Processing Requirements

The performance of index processing operations depends heavily on where the index buffer exists in memory and what type of rendering device is being used. Applications control the memory allocation for index buffers when they are created.

The application can directly write indices to an index buffer allocated in driver-optimal memory. This technique prevents a redundant copy operation later. This technique does not work well if your application reads data back from an index buffer, because read operations done by the host from driver-optimal memory can be very slow. Therefore, if your application needs to read during processing or writes data to the buffer erratically, a system-memory index buffer is a better choice.

Related topics

[Vertex and index buffers](#)

Feedback

Was this page helpful?



Yes



No

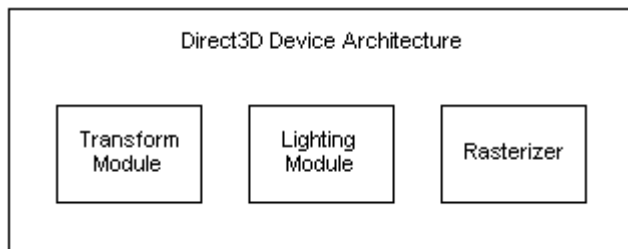
[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Devices

Article • 10/20/2022

A Direct3D device is the rendering component of Direct3D. A device encapsulates and stores the rendering state, performs transformations and lighting operations, and rasterizes an image to a surface.

Architecturally, Direct3D devices contain a transformation module, a lighting module, and a rasterizing module, as the following diagram shows.



Direct3D supports two main types of Direct3D devices:

- A hal device with hardware-accelerated rasterization and shading with both hardware and software vertex processing
- A reference device

These devices are two separate drivers. Software and reference devices are represented by software drivers, and the hal device is represented by a hardware driver. The most common way to take advantage of these devices is to use the hal device for shipping applications, and the reference device for feature testing. These are provided by third parties to emulate particular devices - for example, developmental hardware that has not yet been released.

The Direct3D device that an application creates must correspond to the capabilities of the hardware on which the application is running. Direct3D provides rendering capabilities, either by accessing 3D hardware that is installed in the computer or by emulating the capabilities of 3D hardware in software. Therefore, Direct3D provides devices for both hardware access and software emulation.

Hardware-accelerated devices give much better performance than software devices. The hal device type is available on all Direct3D supported graphic adapters. In most cases, applications target computers that have hardware acceleration and rely on software emulation to accommodate lower-end computers.

With the exception of the reference device, software devices do not always support the same features as a hardware device. Applications should always query for device

capabilities to determine which features are supported.

Because the behavior of the software and reference devices provided with Direct3D 9 is identical to that of the hal device, application code authored to work with the hal device will work with the software or reference devices without modifications. The provided software or reference device behavior is identical to that of the hal device, but the device capabilities do vary, and a particular software device may implement a much smaller set of capabilities.

In this section

Topic	Description
Device types	Direct3D device types include Hardware Abstraction Layer (hal) devices and the reference rasterizer.
Windowed vs. full-screen mode	Direct3D applications can run in either of two modes: windowed or full-screen. In <i>windowed mode</i> , the application shares the available desktop screen space with all running applications. In <i>full-screen mode</i> , the window that the application runs in covers the entire desktop, hiding all running applications (including your development environment).
Lost devices	A Direct3D device can be in either an operational state or a lost state. The <i>operational</i> state is the normal state of the device in which the device runs and presents all rendering as expected. The device makes a transition to the <i>lost</i> state when an event, such as the loss of keyboard focus in a full-screen application, causes rendering to become impossible.
Swap chains	A swap chain is a collection of buffers that are used for displaying frames to the user. Each time an application presents a new frame for display, the first buffer in the swap chain takes the place of the displayed buffer. This process is called <i>swapping</i> or <i>flipping</i> .
Introduction to rasterization rules	Often, the points specified for vertices do not precisely match the pixels on the screen. When this happens, Direct3D applies triangle rasterization rules to decide which pixels apply to a given triangle.

Related topics

[Direct3D Graphics Learning Guide](#)

Device types

Article • 10/20/2022

Direct3D device types include Hardware Abstraction Layer (hal) devices and the reference rasterizer.

HAL Device

The primary device type is the hal device, which supports hardware accelerated rasterization and both hardware and software vertex processing. If the computer on which your application is running is equipped with a display adapter that supports Direct3D, your application should use it for Direct3D operations. Direct3D hal devices implement all or part of the transformation, lighting, and rasterizing modules in hardware.

Applications do not access graphics adapters directly. They call Direct3D functions and methods. Direct3D accesses the hardware through the hal. If the computer that your application is running on supports the hal, it will gain the best performance by using a hal device.

Reference Device

Direct3D supports an additional device type called a reference device or reference rasterizer. Unlike a software device, the reference rasterizer supports every Direct3D feature. This device is intended to be used for debugging purposes and is therefore only available on machines where the DirectX SDK has been installed. Because these features are implemented for accuracy rather than speed and are implemented in software, the results are not very fast. The reference rasterizer does make use of special CPU instructions whenever it can, but it is not intended for retail applications. Use the reference rasterizer only for feature testing or demonstration purposes.

HAL vs. REF Devices

HAL (Hardware Abstraction Layer) devices and REF (REference rasterizer) devices are the two main types of Direct3D device; the first is based around the hardware support, and is very fast but might not support everything; while the second uses no hardware acceleration, so is very slow, but is guaranteed to support the entire set of Direct3D features, in the correct way. In general you'll only ever need to use HAL devices, but if

you're using some advanced feature that your graphics card does not support then you might need to fall back to REF.

The other time you might want to use REF is if the HAL device is producing strange results - that is, you're sure your code is correct, but the result is not what you're expecting. The REF device is guaranteed to behave correctly, so you may wish to test your application on the REF device and see if the strange behavior continues. If it doesn't, it means that either (a) your application is assuming that the graphics card supports something that it doesn't, or (b) it's a driver bug. If it still doesn't work with the REF device, then it's an application bug.

Hardware vs. Software Vertex Processing

Hardware versus Software vertex processing only really applies to HAL devices. When you push vertices through the pipeline, they need to be transformed (by the world, view, and projection matrices in turn) and lit (by D3D's built-in lights) - this processing stage is known as T&L (for Transformation & Lighting). Hardware vertex processing means this is done in hardware, if the hardware supports it; ergo, Software vertex processing is done in software. The general practice is to try creating a Hardware T&L device first, and if that fails try Mixed, and if that fails try Software. (If software fails, give up and exit with an error.)

Related topics

[Devices](#)

Windowed vs. full-screen mode

Article • 10/20/2022

Direct3D applications can run in either of two modes: windowed or full-screen. In *windowed mode*, the application shares the available desktop screen space with all running applications. In *full-screen mode*, the window that the application runs in covers the entire desktop, hiding all running applications (including your development environment). Games typically default to full-screen mode to fully immerse the user in the game by hiding all running applications.

Code differences between full-screen mode and windowed mode are very small.

Because an application running in full-screen mode takes over the screen, debugging the application requires either a separate monitor or the use of a remote debugger. One advantage of a windowed-mode application is that you can step through the code in a debugger without multiple monitors or a remote debugger.

Related topics

[Devices](#)

Lost devices

Article • 10/20/2022

A Direct3D device can be in either an operational state or a lost state. The *operational* state is the normal state of the device in which the device runs and presents all rendering as expected. The device makes a transition to the *lost* state when an event, such as the loss of keyboard focus in a full-screen application, causes rendering to become impossible. The lost state is characterized by the silent failure of all rendering operations, which means that the rendering methods can return success codes even though the rendering operations fail.

By design, the full set of scenarios that can cause a device to become lost is not specified. Some typical examples include loss of focus, such as when the user presses ALT+TAB or when a system dialog is initialized. Devices can also be lost due to a power management event, or when another application assumes full-screen operation. In addition, any failure from resetting a device puts the device into a lost state.

All methods that derive from **IUnknown** are guaranteed to work after a device is lost. After device loss, each function generally has the following three options:

- Fail with a "device lost" error - This means that the application needs to recognize that the device was lost, so that the application identifies that something isn't happening as expected.
- Silently fail, returning S_OK or any other return code - If a function silently fails, the application generally can't distinguish between the result of "success" and a "silent failure."
- Return a return code.

Responding to a Lost Device

A lost device must re-create resources (including video memory resources) after it has been reset. If a device is lost, the application queries the device to see if it can be restored to the operational state. If not, the application waits until the device can be restored.

If the device can be restored, the application prepares the device by destroying all video-memory resources and any swap chains. Resetting is the only procedure that has an effect when a device is lost, and is the only way by which an application can change the device from a lost to an operational state. Reset will fail unless the application releases all resources that are allocated, including render targets and depth-stencil surfaces.

For the most part, the high-frequency calls of Direct3D do not return any information about whether the device has been lost. The application can continue to call rendering methods, without receiving notification of a lost device. Internally, these operations are discarded until the device is reset to the operational state.

Locking Operations

Internally, Direct3D does enough work to ensure that a lock operation will succeed after a device is lost. However, it is not guaranteed that the video-memory resource's data will be accurate during the lock operation. It is guaranteed that no error code will be returned. This allows applications to be written without concern for device loss during a lock operation.

Resources

Resources can consume video memory. Because a lost device is disconnected from the video memory owned by the adapter, it is not possible to guarantee allocation of video memory when the device is lost. As a result, all resource creation methods are implemented to succeed, but do in fact allocate only dummy system memory. Because any video-memory resource must be destroyed before the device is resized, there is no issue of over-allocating video memory. These dummy surfaces allow lock and copy operations to appear to function normally until the application discovers that the device has been lost.

All video memory must be released before a device can be reset from a lost state to an operational state. Other state data is automatically destroyed by the transition to an operational state.

You are encouraged to develop applications with a single code path to respond to device loss. This code path is likely to be similar, if not identical, to the code path taken to initialize the device at startup.

Retrieved Data

Direct3D allows applications to validate texture and render states against single pass rendering by the hardware.

Direct3D also allows applications to copy generated or previously written images from video-memory resources to nonvolatile system-memory resources. Because the source images of such transfers might be lost at any time, Direct3D allows such copy operations to fail when the device is lost.

Copy operations can fail as there is no primary surface when the device is lost. Creating swap chains can also fail because a back buffer can't be created when the device is lost.

Related topics

[Devices](#)

Swap chains

Article • 10/20/2022

A swap chain is a collection of buffers that are used for displaying frames to the user. Each time an application presents a new frame for display, the first buffer in the swap chain takes the place of the displayed buffer. This process is called *swapping* or *flipping*.

A graphics adapter holds a pointer to a surface that represents the image being displayed on the monitor, called a front buffer. As the monitor is refreshed, the graphics card sends the contents of the front buffer to the monitor to be displayed. However, this leads to a "tearing" problem when rendering real-time graphics. The heart of the problem is that monitor refresh rates are very slow in comparison to the rest of the computer. Common refresh rates range from 60 Hz (60 times per second) to 100 Hz.

If your application is updating the front buffer while the monitor is in the middle of a refresh, the image that is displayed will be cut in half with the upper half of the display containing the old image and the lower half containing the new image. This problem is referred to as *tearing*.

Avoiding tearing

Direct3D implements two options to avoid tearing:

- An option to only allow updates of the monitor on the vertical retrace (or vertical sync) operation. A monitor typically refreshes its image by moving a light pen horizontally, zigzagging from the top left of the monitor and ending at the bottom right. When the light pen reaches the bottom, the monitor recalibrates the light pen by moving it back to the upper left so that the process can start again.

This recalibration is called a vertical sync. During a vertical sync, the monitor is not drawing anything, so any update to the front buffer will not be seen until the monitor starts to draw again. The vertical sync is relatively slow; however, not slow enough to render a complex scene while waiting. What is needed to avoid tearing and be able to render complex scenes is a process called back buffering.

- An option to use a technique called back buffering. Back buffering is the process of drawing a scene to an off-screen surface, called a back buffer. Any surface other than the front buffer is called an off-screen surface because it is never directly viewed by the monitor.

By using a back buffer, an application has the freedom to render a scene whenever the system is idle (that is, no windows messages are waiting) without having to consider the monitor's refresh rate. Back buffering brings in an additional complication of how and when to move the back buffer to the front buffer.

Surface flipping

The process of moving the back buffer to the front buffer is called surface flipping. Because the graphics card simply uses a pointer to a surface to represent the front buffer, a simple pointer change is all that is needed to set the back buffer to the front buffer. When an application asks Direct3D to present the back buffer to the front buffer, Direct3D simply "flips" the two surface pointers. The result is that the back buffer is now the new front buffer, and the old front buffer is the new back buffer.

A surface flip is invoked whenever an application asks the Direct3D device to present the back buffer; however, Direct3D can be set up to queue the requests until a vertical sync occurs. This option is referred to as the Direct3D device's presentation interval. The data in the new back buffer may not be reusable, depending on how an application specifies how Direct3D should handle surface flipping.

Surface flipping is key in multimedia, animation, and game software; it is equivalent to the way you can do animation with a pad of paper. On each page, the artist changes the figures slightly, so that when you flip rapidly between sheets, the drawing appears animated.

Related topics

[Devices](#)

Introduction to rasterization rules

Article • 10/20/2022

Often, the points specified for vertices do not precisely match the pixels on the screen. When this happens, Direct3D applies triangle rasterization rules to decide which pixels apply to a given triangle.

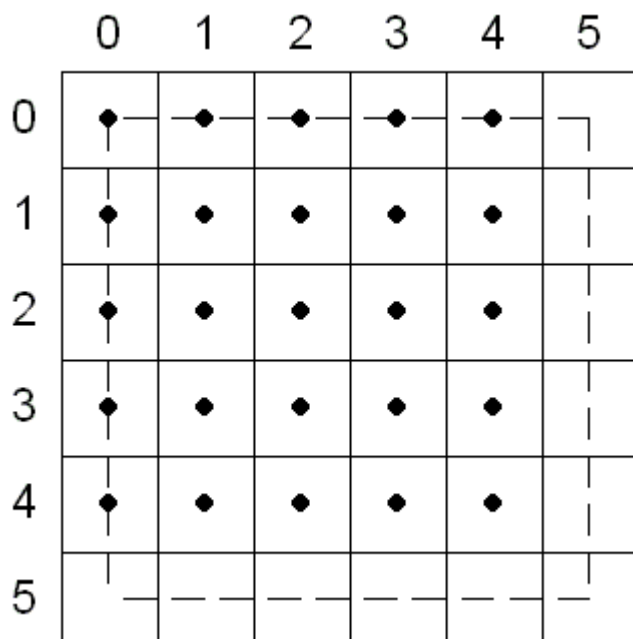
This is a simplified introduction to rasterization rules. For more details, see [Rasterization rules](#). See also [Rasterizer \(RS\) stage](#).

Triangle Rasterization Rules

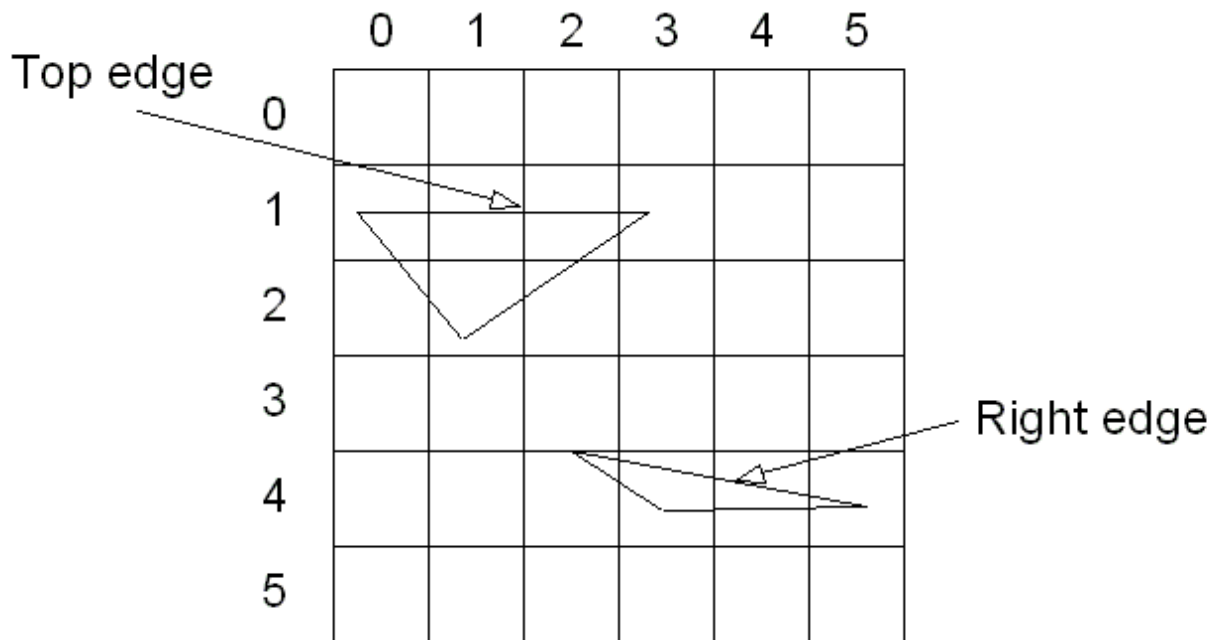
Direct3D uses a top-left filling convention for filling geometry. This is the same convention that is used for rectangles in GDI and OpenGL. In Direct3D, the center of the pixel is the decisive point. If the center is inside a triangle, the pixel is part of the triangle. Pixel centers are at integer coordinates.

This description of triangle-rasterization rules used by Direct3D does not necessarily apply to all available hardware. Your testing may uncover minor variations in the implementation of these rules.

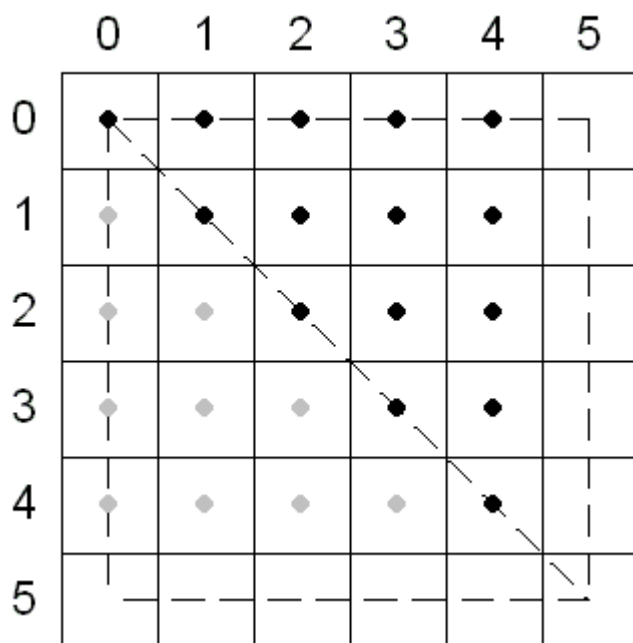
The following illustration shows a rectangle whose upper-left corner is at (0, 0) and whose lower-right corner is at (5, 5). This rectangle fills 25 pixels, just as you would expect. The width of the rectangle is defined as right minus left. The height is defined as bottom minus top.



In the top-left filling convention, *top* refers to the vertical location of horizontal spans, and *left* refers to the horizontal location of pixels within a span. An edge cannot be a top edge unless it is horizontal. In general, most triangles have only left and right edges. The following illustration shows a top edge and a right edge.

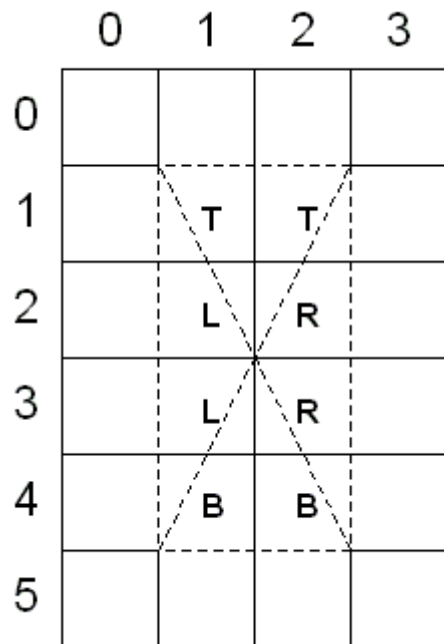


The top-left filling convention determines the action taken by Direct3D when a triangle passes through the center of a pixel. The following illustration shows two triangles, one at (0, 0), (5, 0), and (5, 5), and the other at (0, 5), (0, 0), and (5, 5). The first triangle in this case gets 15 pixels (shown in black), whereas the second gets only 10 pixels (shown in gray) because the shared edge is the left edge of the first triangle.

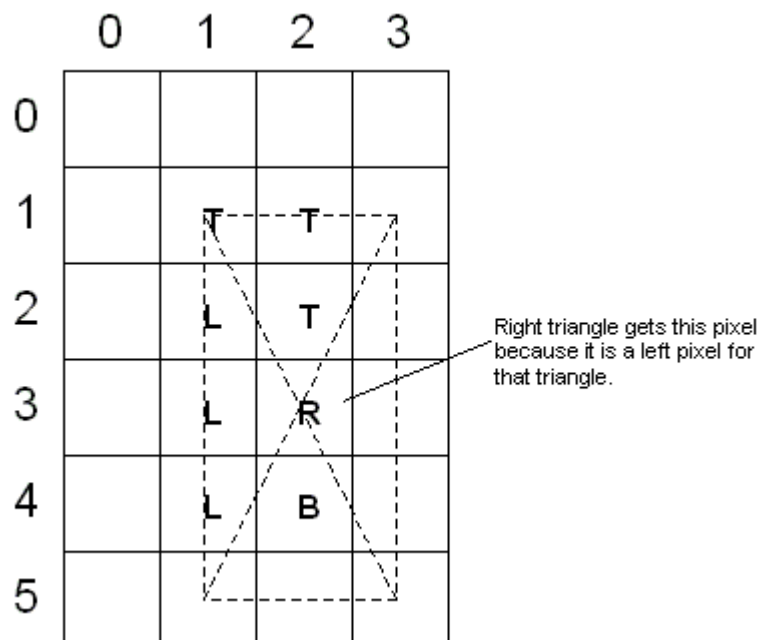


If you define a rectangle with its upper-left corner at (0.5, 0.5) and its lower-right corner at (2.5, 4.5), the center point of this rectangle is at (1.5, 2.5). When the Direct3D rasterizer tessellates this rectangle, the center of each pixel is unambiguously inside

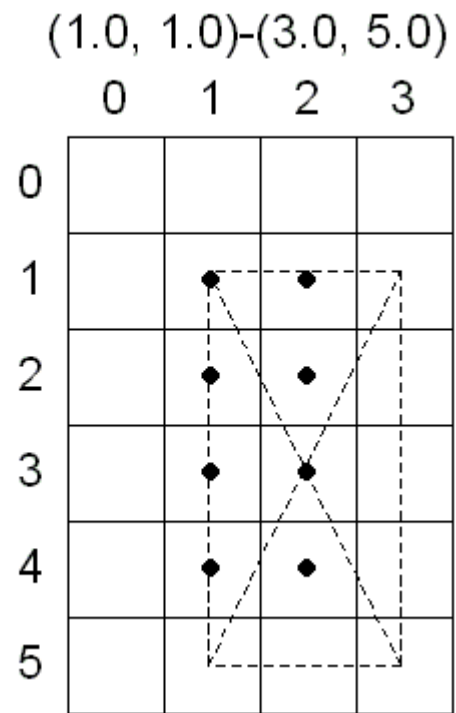
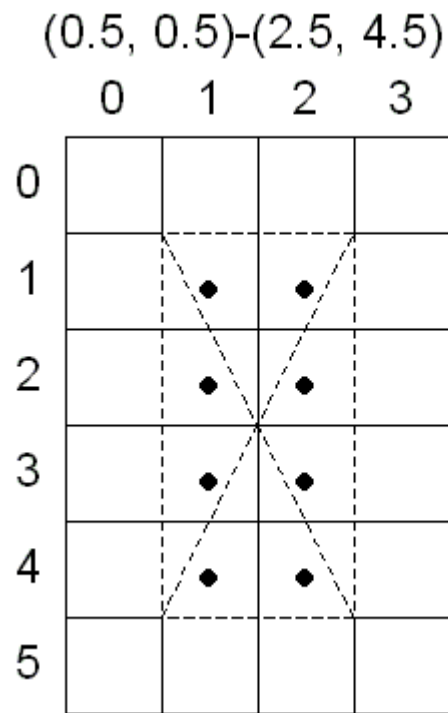
each of the four triangles, and the top-left filling convention is not needed. The following illustration shows this. The pixels in the rectangle are labeled according to the triangle in which Direct3D includes them.



If you move the rectangle in the preceding illustration so that its upper-left corner is at (1.0, 1.0), its lower-right corner at (3.0, 5.0), and its center point at (2.0, 3.0), Direct3D applies the top-left filling convention. Most pixels in this rectangle straddle the border between two or more triangles, as the following illustration shows.



For both rectangles, the same pixels are affected, as shown in the following illustration.



Point and Line Rules

Points are rendered the same as point sprites, which are both rendered as screen-aligned quadrilaterals and thus adhere to the same rules as polygon rendering.

Non-antialiased line rendering rules are exactly the same as those for [GDI lines](#).

Point Sprite Rules

Point sprites and patch primitives are rasterized as if the primitives were first tessellated into triangles and the resulting triangles rasterized.

Related topics

[Devices](#)

[Rasterizer \(RS\) stage](#)

[Rasterization rules](#)

Lighting

Article • 10/20/2022

Lights are used to illuminate objects in a scene. The color of each object vertex is based on the current texture map, vertex colors, and light sources.

Note This section is only for the fixed-function pipeline. Programmable shaders perform all lighting explicitly.

In this section

Topic	Description
Lighting overview	When you use Direct3D lighting, you allow Direct3D to handle the details of illumination for you. Advanced users can perform lighting on their own, if desired.
Light types	The light type property defines which type of light source you're using. There are three types of lights in Direct3D - point lights, spotlights, and directional lights.
Light properties	Light properties describe a light source's type (point, directional, spotlight), attenuation, color, direction, position, and range.
Mathematics of lighting	The Direct3D Light Model covers ambient, diffuse, specular, and emissive lighting. This is enough flexibility to solve a wide range of lighting situations. The total amount of light in a scene is called the <i>global illumination</i> .

Related topics

[Direct3D Graphics Learning Guide](#)

Lighting overview

Article • 10/20/2022

When you use Direct3D lighting, you allow Direct3D to handle the details of illumination for you. Advanced users can perform lighting on their own, if desired.

When lighting is enabled, Direct3D calculates the color of each object vertex based on a combination of the following:

- The texels in an associated texture map.
- The diffuse and specular colors at the vertex, if specified.
- The color and intensity of light produced by light sources in the scene or the scene's ambient light level.

How you work with lighting and materials makes a big difference in the appearance of the rendered scene. Materials define how light reflects off a surface. Direct light and ambient light levels define the light that is reflected. You must use materials to render a scene if lighting is enabled.

Lights are not required to render a scene, but details in a scene rendered without light are not visible. At best, rendering an unlit scene results in a silhouette of the objects in the scene. This is not enough detail for most purposes.

Direct light vs. ambient light

Although both direct and ambient light illuminate objects in a scene, they are independent of one another, they have very different effects, and they require that you work with them in completely different ways.

Direct light strikes the object directly. Direct light always has direction and color, and it is a factor for shading algorithms, such as Gouraud shading. Different types of lights emit direct light in different ways, creating special attenuation effects.

Ambient light is effectively everywhere in a scene. Ambient light is a general level of light that fills an entire scene, regardless of the objects and their locations in that scene. Ambient light has no position or direction, only color and intensity. Each light adds to the overall ambient light in a scene.

Ambient light color takes the form of an RGBA value, where each component is an integer value from 0 to 255. This is unlike most color values in Direct3D.

The red, green, and blue components combine to make the final color of the ambient light. The alpha component controls the transparency of the color. When using hardware acceleration or RGB emulation, the alpha component is ignored.

Direct3D light model vs. nature

In nature, when light is emitted from a source, it is reflected off of hundreds, if not thousands or millions of objects before reaching the user's eye. Each time it is reflected, some light is absorbed by a surface, some is scattered in random directions, and the rest goes on to another surface or to the user's eye. This process continues until the light is reduced to nothing or a user perceives the light.

Obviously, the calculations required to perfectly simulate the natural behavior of light are too time-consuming to use for real-time Direct3D graphics. Therefore, with speed in mind, the Direct3D light model approximates the way light works in the natural world. Direct3D describes light in terms of red, green, and blue components that combine to create a final color.

In Direct3D, when light reflects off a surface, the light color interacts mathematically with the surface itself to create the color eventually displayed on the screen. For specific information about the algorithms Direct3D uses, see [Mathematics of lighting](#).

The Direct3D light model generalizes light into two types: ambient light and direct light. Each has different attributes, and each interacts with the material of a surface in different ways. Ambient light is light that has been scattered so much that its direction and source are indeterminate: it maintains a low level of intensity everywhere. The indirect lighting used by photographers is a good example of ambient light.

Ambient light in Direct3D, as in nature, has no real direction or source, only a color and intensity. In fact, the ambient light level is completely independent of any objects in a scene that generate light. Ambient light does not contribute to specular reflection.

Direct light is the light generated by a source within a scene; it always has color and intensity, and it travels in a specified direction. Direct light interacts with the material of a surface to create specular highlights, and its direction is used as a factor in shading algorithms, including Gouraud shading. When direct light is reflected, it does not contribute to the ambient light level in a scene. The sources in a scene that generate direct light have different characteristics that affect how they illuminate a scene.

Additionally, a polygon's material has properties that affect how that polygon reflects the light it receives. You set a single reflectance trait that describes how the material

reflects ambient light, and you set individual traits to determine the material's specular and diffuse reflectance.

Color values for lights and materials

Direct3D describes color in terms of four components (red, green, blue, and alpha), that combine to make a final color. Each component ranges from 0.0 to 1.0. Although both lights and materials use the same structure to describe color, the values are used a little differently by lights versus materials.

Color values for light sources represent the amount of a particular light component it emits. Because lights don't use an alpha component, only the red, green, and blue components of the color are relevant. You can visualize the three components as the red, green, and blue lenses on a projection television. Each lens might be off (a 0.0 value in the appropriate member), it might be as bright as possible (a 1.0 value), or it might be some level in between.

The colors coming through the lenses combine to make the light's final color. A combination like R(1.0), G(1.0), B(1.0) creates a white light, where R(0.0), G(0.0), B(0.0) doesn't emit light at all. You can make a light that emits only one component, resulting in a pure red, green, or blue light; or, the light could use combinations to emit colors like yellow or purple. You can even set negative color component values to create a "dark light" that actually removes light from a scene. Or, you might set the components to some value larger than 1.0 to create an extremely bright light.

With materials, on the other hand, color values represent how much of a light component is reflected by a surface that is rendered with that material. A material whose color components are R(1.0), G(1.0), B(1.0), A(1.0) reflects all the light that comes its way. Likewise, a material with R(0.0), G(1.0), B(0.0), A(1.0) reflects all the green light that is directed at it. Materials have multiple reflectance values to create various types of effects.

See [Light types](#) and [Light properties](#).

Related topics

[Lights and materials](#)

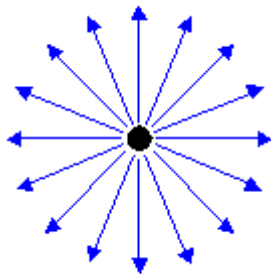
Light types

Article • 10/20/2022

The light type property defines which type of light source you're using. There are three types of lights in Direct3D - point lights, spotlights, and directional lights. Each type illuminates objects in a scene differently, with varying levels of computational overhead.

Point Light

Point lights have color and position within a scene, but no single direction. They give off light equally in all directions, as shown in the following illustration.



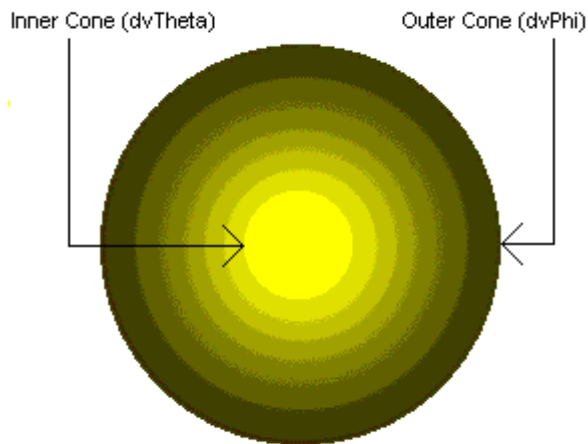
A light bulb is a good example of a point light. Point lights are affected by attenuation and range, and illuminate a mesh on a vertex-by-vertex basis. During lighting, Direct3D uses the point light's position in world space and the coordinates of the vertex being lit to derive a vector for the direction of the light, and the distance that the light has traveled. Both are used, along with the vertex normal, to calculate the contribution of the light to the illumination of the surface.

Directional Light

Directional lights have only color and direction, not position. They emit parallel light. This means that all light generated by directional lights travels through a scene in the same direction. Imagine a directional light as a light source at near infinite distance, such as the sun. Directional lights are not affected by attenuation or range, so the direction and color you specify are the only factors considered when Direct3D calculates vertex colors. Because of the small number of illumination factors, these are the least computationally intensive lights to use.

SpotLight

Spotlights have color, position, and direction in which they emit light. Light emitted from a spotlight is made up of a bright inner cone and a larger outer cone, with the light intensity diminishing between the two, as shown in the following illustration.

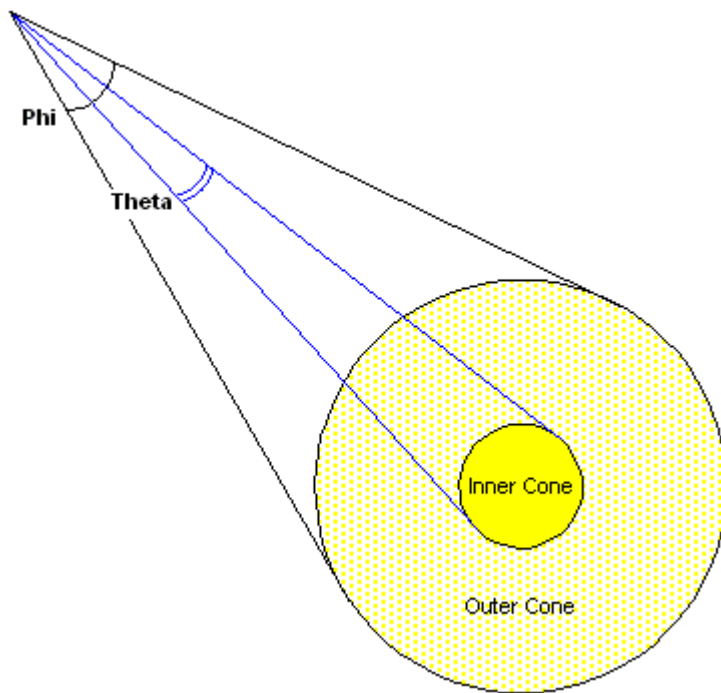


Spotlights are affected by falloff, attenuation, and range. These factors, as well as the distance light travels to each vertex, are figured in when computing lighting effects for objects in a scene. Computing these effects for each vertex makes spotlights the most computationally time-consuming of all lights in Direct3D.

Falloff, Theta, and Phi values are used only by spotlights. These values control how large or small a spotlight object's inner and outer cones are, and how light decreases between them.

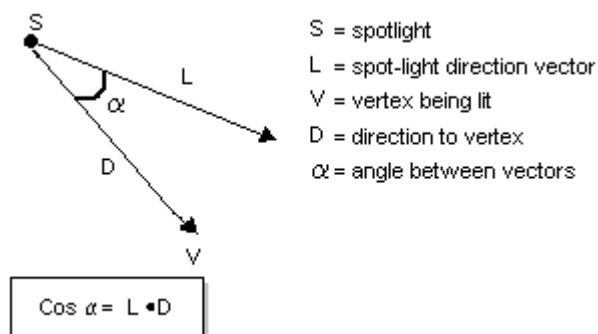
Theta is the radian angle of the spotlight's inner cone, and the Phi value is the angle for the outer cone of light. Falloff controls how light intensity decreases between the outer edge of the inner cone and the inner edge of the outer cone. Most applications set Falloff to 1.0 to create falloff that occurs evenly between the two cones, but you can set other values as needed.

The following illustration shows the relationship between these values and how they can affect a spotlight's inner and outer cones of light.



Spotlights emit a cone of light that has two parts: a bright inner cone and an outer cone. Light is brightest in the inner cone and isn't present outside the outer cone, with light intensity attenuating between the two areas. This type of attenuation is commonly referred to as falloff.

The amount of light a vertex receives is based on the vertex's location in the inner or outer cones. Direct3D computes the dot product of the spotlight's direction vector (L) and the vector from the light to the vertex (D). This value is equal to the cosine of the angle between the two vectors, and serves as an indicator of the vertex's position that can be compared to the light's cone angles to determine where the vertex might lie in the inner or outer cones. The following illustration provides a graphical representation of the association between these two vectors.



The system compares this value to the cosine of the spotlight's inner and outer cone angles. The light's Theta and Phi values represent the total cone angles for the inner and outer cones. Because the attenuation occurs as the vertex becomes more distant from the center of illumination (rather than across the total cone angle), the runtime divides these cone angles in half before calculating their cosines.

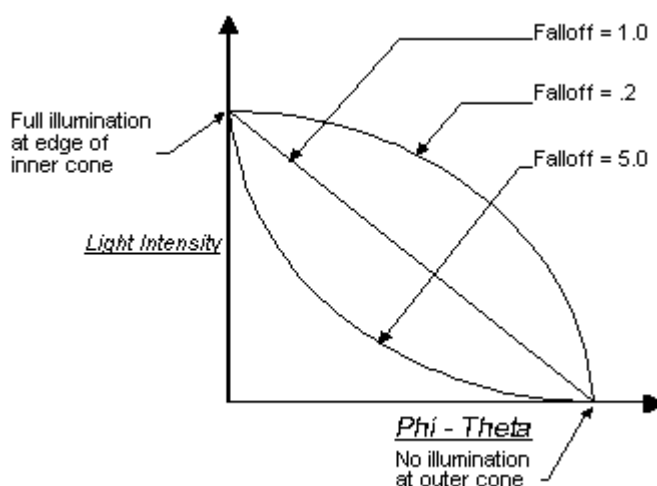
If the dot product of vectors L and D is less than or equal to the cosine of the outer cone angle, the vertex lies beyond the outer cone and receives no light. If the dot product of L and D is greater than the cosine of the inner cone angle, then the vertex is within the inner cone and receives the maximum amount of light, still considering attenuation over distance. If the vertex is somewhere between the two regions, then falloff is calculated with the following equation.

$$I_f = \left(\frac{\cos(\alpha) - \cos(\phi/2)}{\cos(\theta/2) - \cos(\phi/2)} \right)^p$$

where:

- I_f is light intensity after falloff
- α is the angle between vectors L and D
- θ is the inner cone angle
- ϕ is the outer cone angle
- p is the falloff

This formula generates a value between 0.0 and 1.0 that scales the light's intensity at the vertex to account for falloff. Attenuation as a factor of the vertex's distance from the light is also applied. The following graph shows how different falloff values can affect the falloff curve.



The effect of various falloff values on the actual lighting is subtle, and a small performance penalty is incurred by shaping the falloff curve with falloff values other than 1.0. For these reasons, this value is typically set to 1.0.

Related topics

[Lights and materials](#)

Light properties

Article • 10/20/2022

Light properties describe a light source's type (point, directional, spotlight), attenuation, color, direction, position, and range. Depending on the type of light being used, a light can have properties for attenuation and range, or for spotlight effects. Not all types of lights use all properties.

The position, range, and attenuation properties define a light's location in world space and how the light it emits behaves over distance.

Light Attenuation

Attenuation controls how a light's intensity decreases toward the maximum distance specified by the range property. Three floating point values are sometimes used to represent light attenuation: Attenuation0, Attenuation1, and Attenuation2. These floating-point values ranging from 0.0 through infinity, controlling a light's attenuation. Some applications set the Attenuation1 member to 1.0 and the others to 0.0, resulting in light intensity that changes as $1 / D$, where D is the distance from the light source to the vertex. The maximum light intensity is at the source, decreasing to $1 / (\text{Light Range})$ at the light's range.

Though typically, an application sets Attenuation0 to 0.0, Attenuation1 to a constant value, and Attenuation2 to 0.0, varying light effects can be achieved by changing this. You can combine attenuation values to get more complex attenuation effects. Or, you might set them to values outside the normal range to create even stranger attenuation effects. Negative attenuation values, however, are not allowed. See [Attenuation and spotlight factor](#).

Light Color

Lights in Direct3D emit three colors that are used independently in the system's lighting computations: a diffuse color, an ambient color, and a specular color. Each is incorporated by the Direct3D lighting module, interacting with a counterpart from the current material, to produce a final color used in rendering. The diffuse color interacts with the diffuse reflectance property of the current material, the specular color with the material's specular reflectance property, and so on. For specifics about how Direct3D applies these colors, see [Mathematics of lighting](#).

In a Direct3D application, typically there are three color values - Diffuse, Ambient, and Specular that defines the color being emitted.

The type of color that applies most heavily to the system's computations is the diffuse color. The most common diffuse color is white (R:1.0 G:1.0 B:1.0), but you can create colors as needed to achieve desired effects. For example, you could use red light for a fireplace, or you could use green light for a traffic signal set to "Go."

Generally, you set the light color components to values between 0.0 and 1.0, inclusive, but this isn't a requirement. For example, you might set all the components to 2.0, creating a light that is "brighter than white." This type of setting can be especially useful when you use attenuation settings other than constant.

Although Direct3D uses RGBA values for lights, the alpha color component is not used.

Usually material colors are used for lighting. However, you can specify that material colors-emissive, ambient, diffuse, and specular-are to be overridden by diffuse or specular vertex colors.

The alpha/transparency value always comes only from the diffuse color's alpha channel.

The fog value always comes only from the specular color's alpha channel.

Light Direction

A light's direction property determines the direction that the light emitted by the object travels, in world space. Direction is used only by directional lights and spotlights, and is described with a vector.

Set the light direction as a vector. Direction vectors are described as distances from a logical origin, regardless of the light's position in a scene. Therefore, a spotlight that points straight into a scene - along the positive z-axis - has a direction vector of $\langle 0, 0, 1 \rangle$ no matter where its position is defined to be. Similarly, you can simulate sunlight shining directly on a scene by using a directional light whose direction is $\langle 0, -1, 0 \rangle$. You don't have to create lights that shine along the coordinate axes; you can mix and match values to create lights that shine at more interesting angles.

Although you don't need to normalize a light's direction vector, always be sure that it has magnitude. In other words, don't use a $\langle 0, 0, 0 \rangle$ direction vector.

Light Position

Light position is described using a vector structure. The x, y, and z coordinates are assumed to be in world space. Directional lights are the only type of light that don't use the position property.

Light Range

A light's range property determines the distance, in world space, at which meshes in a scene no longer receive light emitted by that object. Directional lights don't use the range property.

Related topics

[Lights and materials](#)

Mathematics of lighting

Article • 10/20/2022

The Direct3D Light Model covers ambient, diffuse, specular, and emissive lighting. This is enough flexibility to solve a wide range of lighting situations. The total amount of light in a scene is called the *global illumination*.

The global illumination is computed as follows:

C++

```
global_illumination = ambient_lighting + diffuse_lighting +  
specular_lighting + emissive_lighting;
```

[Ambient lighting](#) is constant lighting. Ambient lighting is constant in all directions and it colors all pixels of an object the same. It is fast to calculate but leaves objects looking flat and unrealistic.

[Diffuse lighting](#) depends on both the light direction and the object surface normal. Diffuse lighting varies across the surface of an object as a result of the changing light direction and the changing surface normal vector. It takes longer to calculate diffuse lighting because it changes for each object vertex, however the benefit of using it is that it shades objects and gives them three-dimensional (3D) depth.

[Specular lighting](#) identifies the bright specular highlights that occur when light hits an object surface and reflects back toward the camera. Specular lighting is more intense than diffuse light and falls off more rapidly across the object surface. It takes longer to calculate specular lighting than diffuse lighting, however the benefit of using it is that it adds significant detail to a surface.

[Emissive lighting](#) is light that is emitted by an object; for example, a glow. Emission makes a rendered object appear to be self-luminous. Emission affects an object's color and can, for example, make a dark material brighter and take on part of the emitted color.

Realistic lighting can be accomplished by applying each of these types of lighting to a 3D scene. The values calculated for ambient, emissive, and diffuse components are output as the diffuse vertex color; the value for the specular lighting component is output as the specular vertex color. Ambient, diffuse, and specular light values can be affected by a given light's attenuation and spotlight factor. See [Attenuation and spotlight factor](#).

To achieve a more realistic lighting effect, you add more lights; however, the scene takes longer to render. To achieve all the effects a designer wants, some games use more CPU power than is commonly available. In this case, it is typical to reduce the number of lighting calculations to a minimum by using lighting maps and environment maps to add lighting to a scene while using texture maps.

Lighting is computed in the camera space. See [Camera space transformations](#).

Optimized lighting can be computed in model space, when special conditions exist: normal vectors are already normalized, vertex blending is not needed, and transformation matrices are orthogonal.

All lighting computations are made in model space by transforming the light source's position and direction, along with the camera position, to model space using the inverse of the world matrix. As a result, if the world or view matrices introduce non-uniform scaling, the resultant lighting might be inaccurate.

In this section

Topic	Description
Ambient lighting	Ambient lighting provides constant lighting for a scene. It lights all object vertices the same because it is not dependent on any other lighting factors such as vertex normals, light direction, light position, range, or attenuation. Ambient lighting is constant in all directions and it colors all pixels of an object the same. It is fast to calculate but leaves objects looking flat and unrealistic.
Diffuse lighting	<i>Diffuse lighting</i> depends on both the light direction and the object surface normal. Diffuse lighting varies across the surface of an object as a result of the changing light direction and the changing surface normal vector. It takes longer to calculate diffuse lighting because it changes for each object vertex, however the benefit of using it is that it shades objects and gives them three-dimensional (3D) depth.

Topic	Description
Specular lighting	<i>Specular lighting</i> identifies the bright specular highlights that occur when light hits an object surface and reflects back toward the camera. Specular lighting is more intense than diffuse light and falls off more rapidly across the object surface. It takes longer to calculate specular lighting than diffuse lighting, however the benefit of using it is that it adds significant detail to a surface.
Emissive lighting	<i>Emissive lighting</i> is light that is emitted by an object; for example, a glow. Emission makes a rendered object appear to be self-luminous. Emission affects an object's color and can, for example, make a dark material brighter and take on part of the emitted color.
Camera space transformations	Vertices in the camera space are computed by transforming the object vertices with the world view matrix.
Attenuation and spotlight factor	The diffuse and specular lighting components of the global illumination equation contain terms that describe light attenuation and the spotlight cone.

Related topics

[Lights and materials](#)

Ambient lighting

Article • 10/20/2022

Ambient lighting provides constant lighting for a scene. It lights all object vertices the same because it is not dependent on any other lighting factors such as vertex normals, light direction, light position, range, or attenuation. Ambient lighting is constant in all directions and it colors all pixels of an object the same. It is fast to calculate but leaves objects looking flat and unrealistic.

Ambient lighting is the fastest type of lighting but it produces the least realistic results. Direct3D contains a single global ambient light property that you can use without creating any light. Alternatively, you can set any light object to provide ambient lighting.

The ambient lighting for a scene is described by the following equation.

$$\text{Ambient Lighting} = C_a * [G_a + \sum(\text{Atten}_i * \text{Spot}_i * L_{ai})]$$

Where:

Parameter	Default value	Type	Description
C_a	(0,0,0,0)	D3DCOLORVALUE	Material ambient color
G_a	(0,0,0,0)	D3DCOLORVALUE	Global ambient color
Atten_i	(0,0,0,0)	D3DCOLORVALUE	Light attenuation of the ith light. See Attenuation and spotlight factor .
Spot_i	(0,0,0,0)	D3DVECTOR	Spotlight factor of the ith light. See Attenuation and spotlight factor .
sum	N/A	N/A	Sum of the ambient light
L_{ai}	(0,0,0,0)	D3DVECTOR	Light ambient color of the ith light

The value for C_a is either:

- vertex color1, if AMBIENTMATERIALSOURCE = D3DMCS_COLOR1, and the first vertex color is supplied in the vertex declaration.
- vertex color2, if AMBIENTMATERIALSOURCE = D3DMCS_COLOR2, and the second vertex color is supplied in vertex declaration.
- material ambient color.

Note If either AMBIENTMATERIALSOURCE option is used, and the vertex color is not provided, then the material ambient color is used.

To use the material ambient color, use SetMaterial as shown in the example code below.

G_a is the global ambient color. It is set using SetRenderState(D3DRS_AMBIENT). There is one global ambient color in a Direct3D scene. This parameter is not associated with a Direct3D light object.

L_{ai} is the ambient color of the i th light in the scene. Each Direct3D light has a set of properties, one of which is the ambient color. The term, $\text{sum}(L_{ai})$ is a sum of all the ambient colors in the scene.

Example

In this example, the object is colored using the scene ambient light and a material ambient color.

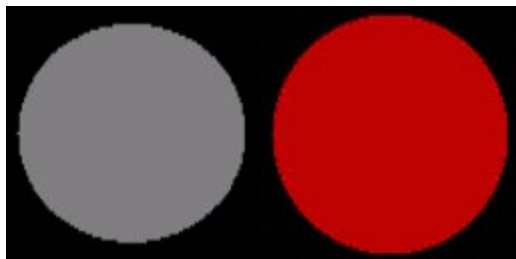
C++

```
#define GRAY_COLOR 0x00bfbfbf

Ambient.r = 0.75f;
Ambient.g = 0.0f;
Ambient.b = 0.0f;
Ambient.a = 0.0f;
```

According to the equation, the resulting color for the object vertices is a combination of the material color and the light color.

The following two illustrations show the material color, which is gray, and the light color, which is bright red.



The resulting scene is shown in the following illustration. The only object in the scene is a sphere. Ambient light lights all object vertices with the same color. It is not dependent

on the vertex normal or the light direction. As a result, the sphere looks like a 2D circle because there is no difference in shading around the surface of the object.



To give objects a more realistic look, apply diffuse or specular lighting in addition to ambient lighting.

Related topics

[Mathematics of lighting](#)

Diffuse lighting

Article • 10/20/2022

Diffuse lighting depends on both the light direction and the object surface normal. Diffuse lighting varies across the surface of an object as a result of the changing light direction and the changing surface normal vector. It takes longer to calculate diffuse lighting because it changes for each object vertex, however the benefit of using it is that it shades objects and gives them three-dimensional (3D) depth.

After adjusting the light intensity for any attenuation effects, the lighting engine computes how much of the remaining light reflects from a vertex, given the angle of the vertex normal and the direction of the incident light. The lighting engine skips to this step for directional lights because they do not attenuate over distance. The system considers two reflection types, diffuse and specular, and uses a different formula to determine how much light is reflected for each.

After calculating the amounts of light reflected, Direct3D applies these new values to the diffuse and specular reflectance properties of the current material. The resulting color values are the diffuse and specular components that the rasterizer uses to produce Gouraud shading and specular highlighting.

Diffuse lighting is described by the following equation.

$$\text{Diffuse Lighting} = \text{sum}[C_d * L_d * (N \cdot L_{\text{dir}}) * \text{Atten} * \text{Spot}]$$

Parameter	Default value	Type	Description
sum	N/A	N/A	Summation of each light's diffuse component.
C_d	(0,0,0,0)	D3DCOLORVALUE	Diffuse color.
L_d	(0,0,0,0)	D3DCOLORVALUE	Light diffuse color.
N	N/A	D3DVECTOR	Vertex normal
L_{dir}	N/A	D3DVECTOR	Direction vector from object vertex to the light.
Atten	N/A	FLOAT	Light attenuation. See Attenuation and spotlight factor .
Spot	N/A	FLOAT	Spotlight factor. See Attenuation and spotlight factor .

To calculate the attenuation (Atten) or the spotlight characteristics (Spot), see [Attenuation and spotlight factor](#).

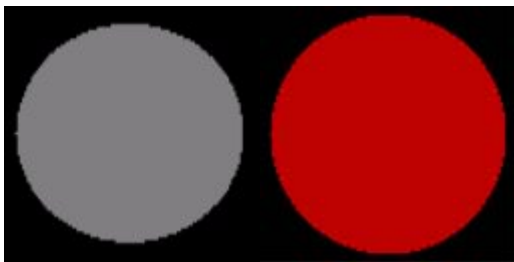
Diffuse components are clamped to be from 0 to 255, after all lights are processed and interpolated separately. The resulting diffuse lighting value is a combination of the ambient, diffuse and emissive light values.

Example

In this example, the object is colored using the light diffuse color and a material diffuse color.

According to the equation, the resulting color for the object vertices is a combination of the material color and the light color.

The following two illustrations show the material color, which is gray, and the light color, which is bright red.



The resulting scene is shown in the following illustration. The only object in the scene is a sphere. The diffuse lighting calculation takes the material and light diffuse color and modifies it by the angle between the light direction and the vertex normal using the dot product. As a result, the backside of the sphere gets darker as the surface of the sphere curves away from the light.



Combining the diffuse lighting with the ambient lighting from the previous example shades the entire surface of the object. The ambient light shades the entire surface and the diffuse light helps reveal the object's 3D shape, as shown in the following illustration.



Diffuse lighting is more intensive to calculate than ambient lighting. Because it depends on the vertex normals and light direction, you can see the objects geometry in 3D space, which produces a more realistic lighting than ambient lighting. You can use specular highlights to achieve a more realistic look.

Related topics

[Mathematics of lighting](#)

Specular lighting

Article • 10/20/2022

Specular lighting identifies the bright specular highlights that occur when light hits an object surface and reflects back toward the camera. Specular lighting is more intense than diffuse light and falls off more rapidly across the object surface. It takes longer to calculate specular lighting than diffuse lighting, however the benefit of using it is that it adds significant detail to a surface.

Modeling specular reflection requires that the system know in what direction light is traveling and the direction to the viewer's eye. The system uses a simplified version of the Phong specular-reflection model, which employs a halfway vector to approximate the intensity of specular reflection.

The default lighting state does not calculate specular highlights.

Specular Lighting Equation

Specular Lighting is described by the following equation.

$$\text{Specular Lighting} = C_s * \text{sum}[L_s * (N \cdot H)^P * \text{Atten} * \text{Spot}]$$

The variables, their types, and their ranges are as follows:

Parameter	Default value	Type	Description
C_s	(0,0,0,0)	Red, green, blue, and alpha transparency (floating-point values)	Specular color.
sum	N/A	N/A	Summation of each light's specular component.
N	N/A	3D vector (x, y, and z floating-point values)	Vertex normal.
H	N/A	3D vector (x, y, and z floating-point values)	Half way vector. See the section on the halfway vector.
P	0.0	Floating point	Specular reflection power. Range is 0 to +infinity
L_s	(0,0,0,0)	Red, green, blue, and alpha transparency (floating-point values)	Light specular color.

Parameter	Default value	Type	Description
Atten	N/A	Floating point	Light attenuation value. See Attenuation and spotlight factor .
Spot	N/A	Floating point	Spotlight factor. See Attenuation and spotlight factor .

The value for C_s is either:

- vertex color 1, if the specular material source is the diffuse vertex color, and the first vertex color is supplied in the vertex declaration.
- vertex color 2, if specular material source is the specular vertex color, and the second vertex color is supplied in the vertex declaration.
- material specular color

Note If either specular material source option is used and the vertex color is not provided, then the material specular color is used.

Specular components are clamped to be from 0 to 255, after all lights are processed and interpolated separately.

The Halfway Vector

The halfway vector (H) exists midway between two vectors: the vector from an object vertex to the light source, and the vector from an object vertex to the camera position. Direct3D provides two ways to compute the halfway vector. When camera-relative specular highlights is enabled (instead of orthogonal specular highlights), the system calculates the halfway vector using the position of the camera and the position of the vertex, along with the light's direction vector. The following formula illustrates this.

$$H = \text{norm}(\text{norm}(C_p - V_p) + L_{\text{dir}})$$

Parameter	Default value	Type	Description
-----------	---------------	------	-------------

Parameter	Default value	Type	Description
C_p	N/A	3D vector (x, y, and z floating-point values)	Camera position.
V_p	N/A	3D vector (x, y, and z floating-point values)	Vertex position.
L_{dir}	N/A	3D vector (x, y, and z floating-point values)	Direction vector from vertex position to the light position.

Determining the halfway vector in this manner can be computationally intensive. As an alternative, using orthogonal specular highlights (instead of camera-relative specular highlights) instructs the system to act as though the viewpoint is infinitely distant on the z-axis. This is reflected in the following formula.

$$H = \text{norm}((0,0,1) + L_{dir})$$

This setting is less computationally intensive, but much less accurate, so it is best used by applications that use orthogonal projection.

Example

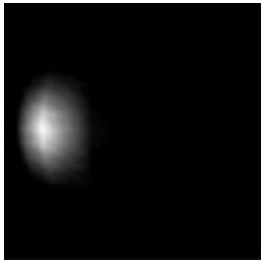
In this example, the object is colored using the scene specular light color and a material specular color.

According to the equation, the resulting color for the object vertices is a combination of the material color and the light color.

The following two illustration show the specular material color, which is gray, and the specular light color, which is white.



The resulting specular highlight is shown in the following illustration.



Combining the specular highlight with the ambient and diffuse lighting produces the following illustration. With all three types of lighting applied, this more clearly resembles a realistic object.



Specular lighting is more intensive to calculate than diffuse lighting. It is typically used to provide visual clues about the surface material. The specular highlight varies in size and color with the material of the surface.

Related topics

[Mathematics of lighting](#)

Emissive lighting

Article • 10/20/2022

Emissive lighting is light that is emitted by an object; for example, a glow. Emission makes a rendered object appear to be self-luminous. Emission affects an object's color and can, for example, make a dark material brighter and take on part of the emitted color.

Emissive lighting is described by a single term.

Emissive Lighting = C_e

where:

Parameter	Default value	Type	Description
C_e	(0,0,0,0)	Red, green, blue, and alpha transparency (all floating-point values)	Emissive color.

The value for C_e is either color 1 or color 2. If the vertex color is not provided, the material emissive color is used.

Example

In this example, the object is colored using the scene ambient light and a material ambient color.

According to the equation, the resulting color for the object vertices is the material color.

The following illustration shows the material color, which is green. Emissive light lights all object vertices with the same color. It is not dependent on the vertex normal or the light direction. As a result, the sphere looks like a 2D circle because there is no difference in shading around the surface of the object.



The following illustration shows how the emissive light blends with the other three types of lights. On the right side of the sphere, there is a blend of the green emissive and the red ambient light. On the left side of the sphere, the green emissive light blends with red ambient and diffuse light producing a red gradient. The specular highlight is white in the center and creates a yellow ring as the specular light value falls off sharply leaving the ambient, diffuse and emissive light values which blend together to make yellow.



Related topics

[Mathematics of lighting](#)

Camera space transformations

Article • 10/20/2022

Vertices in the camera space are computed by transforming the object vertices with the world view matrix.

$$V = V * \text{wvMatrix}$$

Vertex normals, in camera space, are computed by transforming the object normals with the inverse transpose of the world view matrix. The world view matrix may or may not be orthogonal.

$$N = N * (\text{wvMatrix}^{-1})^T$$

The matrix inversion and matrix transpose operate on a 4x4 matrix. The multiply combines the normal with the 3x3 portion of the resulting 4x4 matrix.

If the render state is set to normalize normals, vertex normal vectors are normalized after transformation to camera space as follows:

$$N = \text{norm}(N)$$

Light position in camera space is computed by transforming the light source position with the view matrix.

$$L_p = L_p * \text{vMatrix}$$

The direction to the light in camera space for a directional light is computed by multiplying the light source direction by the view matrix, normalizing, and negating the result.

$$L_{\text{dir}} = -\text{norm}(L_{\text{dir}} * \text{wvMatrix})$$

For a point light and a spotlight, the direction to light is computed as follows:

$L_{\text{dir}} = \text{norm}(V - L_p)$, where the parameters are defined in the following table.

Parameter	Default value	Type	Description
L_{dir}	N/A	3D vector (x, y, and z floating-point values)	Direction vector from object vertex to the light
V	N/A	3D vector (x, y, and z floating-point values)	Vertex position in camera space

Parameter	Default value	Type	Description
wwMatrix	Identity	4x4 matrix of floating-point values	Composite matrix containing the world and view transforms
N	N/A	3D vector (x, y, and z floating-point values)	Vertex normal
L_p	N/A	3D vector (x, y, and z floating-point values)	Light position in camera space
vMatrix	Identity	4x4 matrix of floating-point values	Matrix containing the view transform

Related topics

[Mathematics of lighting](#)

Attenuation and spotlight factor

Article • 10/20/2022

The diffuse and specular lighting components of the global illumination equation contain terms that describe light attenuation and the spotlight cone. These terms are described below.

Attenuation

The attenuation of a light depends on the type of light and the distance between the light and the vertex position. To calculate attenuation, use one of the following equations.

$$\text{Atten} = 1 / (\text{att0}_i + \text{att1}_i * d + \text{att2}_i * d^2)$$

Where:

Parameter	Default value	Type	Description	Range
att0 _i	0.0	Floating point	Constant attenuation factor	0 to +infinity
att1 _i	0.0	Floating point	Linear attenuation factor	0 to +infinity
att2 _i	0.0	Floating point	Quadratic attenuation factor	0 to +infinity
d	N/A	Floating point	Distance from vertex position to light position	N/A

- Atten = 1, if the light is a directional light.
- Atten = 0, if the distance between the light and the vertex exceeds the light's range.

The distance between the light and the vertex position is always positive.

$$d = |L_{\text{dir}}|$$

Where:

Parameter	Default value	Type	Description
L _{dir}	N/A	3D vector with x, y, and z floating-point values	Direction vector from vertex position to the light position

If d is greater than the light's range, Direct3D makes no further attenuation calculations and applies no effects from the light to the vertex.

The attenuation constants act as coefficients in the formula - you can produce a variety of attenuation curves by making simple adjustments to them. You can set Attenuation1 to 1.0 to create a light that doesn't attenuate but is still limited by range, or you can experiment with different values to achieve various attenuation effects.

The attenuation at the maximum range of the light is not 0.0. To prevent lights from suddenly appearing when they are at the light range, an application can increase the light range. Or, the application can set up attenuation constants so that the attenuation factor is close to 0.0 at the light range. The attenuation value is multiplied by the red, green, and blue components of the light's color to scale the light's intensity as a factor of the distance light travels to a vertex.

Spotlight Factor

The following equation specifies the spotlight factor.

$$spot_i = \begin{cases} 1 & \text{for non-spotlights or if } rho_i > \cos(\frac{theta_i}{2}) \\ 0 & \text{if } rho_i \leq \cos(\frac{phi_i}{2}) \\ \left[\frac{rho_i - \cos(\frac{phi_i}{2})}{\cos(\frac{theta_i}{2}) - \cos(\frac{phi_i}{2})} \right]^{falloff} & \text{otherwise} \end{cases}$$

Parameter	Default value	Type	Description	Range
rho _i	N/A	Floating point	cosine(angle) for spotlight i	N/A
phi _i	0.0	Floating point	Penumbra angle of spotlight i in radians	[theta _i , pi)

Parameter	Default value	Type	Description	Range
θ_i	0.0	Floating point	Umbra angle of spotlight i in radians	$[0, \pi)$
falloff	0.0	Floating point	Falloff factor	$(-\infty, +\infty)$

Where:

$$\rho = \text{norm}(\mathbf{L}_{\text{dcs}}) \cdot \text{norm}(\mathbf{L}_{\text{dir}})$$

and:

Parameter	Default value	Type	Description
$\mathbf{L}_{\text{dcs}}$	N/A	3D vector with x, y, and z floating-point values	The negative of the light direction in camera space
$\mathbf{L}_{\text{dir}}$	N/A	3D vector with x, y, and z floating-point values	Direction vector from vertex position to the light position

After computing the light attenuation, to calculate the diffuse and specular components for the vertex, Direct3D also considers spotlight effects if applicable, the angle that the light reflects from a surface, and the reflectance of the current material. In [Light types](#), see "Spotlight".

Related topics

[Mathematics of lighting](#)

Depth and stencil buffers

Article • 10/20/2022

A *depth buffer* stores depth information to control which areas of polygons are rendered rather than hidden from view. A *stencil buffer* is used to mask pixels in an image, to produce special effects, including compositing; decaling; dissolves, fades, and swipes; outlines and silhouettes; and two-sided stencil.

In this section

Topic	Description
Depth buffers	A <i>depth buffer</i> , or <i>z-buffer</i> , stores depth information to control which areas of polygons are rendered rather than hidden from view.
Stencil buffers	A <i>stencil buffer</i> is used to mask pixels in an image, to produce special effects. The mask controls whether the pixel is drawn or not. These special effects include compositing; decaling; dissolves, fades, and swipes; outlines and silhouettes; and two-sided stencil.

Related topics

[Direct3D Graphics Learning Guide](#)

Depth buffers

Article • 10/20/2022

A *depth buffer*, or *z-buffer*, stores depth information to control which areas of polygons are rendered rather than hidden from view.

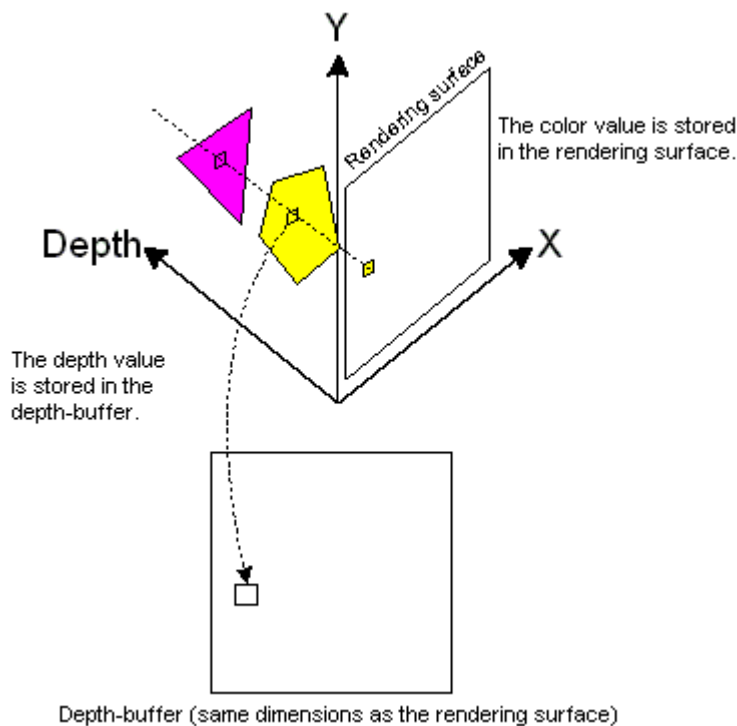
Overview

A depth buffer, often called a z-buffer or a w-buffer, is a property of the device that stores depth information to be used by Direct3D. When Direct3D renders a scene to a target surface, it can use the memory in an associated depth-buffer surface as a workspace to determine how the pixels of rasterized polygons occlude one another. Direct3D uses an off-screen Direct3D surface as the target to which final color values are written. The depth-buffer surface that is associated with the render-target surface is used to store depth information that tells Direct3D how deep each visible pixel is in the scene.

When a scene is rasterized with depth buffering enabled, each point on the rendering surface is tested. The values in the depth buffer can be a point's z-coordinate or its homogeneous w-coordinate - from the point's (x,y,z,w) location in projection space. A depth buffer that uses z values is often called a z-buffer, and one that uses w values is called a w-buffer. Each type of depth buffer has advantages and disadvantages, which are discussed later.

At the beginning of the test, the depth value in the depth buffer is set to the largest possible value for the scene. The color value on the rendering surface is set to either the background color value or the color value of the background texture at that point. Each polygon in the scene is tested to see if it intersects with the current coordinate (x,y) on the rendering surface.

If it does intersect, the depth value - which will be the z coordinate in a z-buffer, and the w coordinate in a w-buffer - at the current point is tested to see if it is smaller than the depth value stored in the depth buffer. If the depth of the polygon value is smaller, it is stored in the depth buffer and the color value from the polygon is written to the current point on the rendering surface. If the depth value of the polygon at that point is larger, the next polygon in the list is tested. This process is shown in the following diagram.



Buffering techniques

Although most applications don't use this feature, you can change the comparison that Direct3D uses to determine which values are placed in the depth buffer and subsequently the render-target surface. On some hardware, changing the compare function may disable hierarchical z testing.

Nearly all accelerators on the market support z-buffering, making z-buffers the most common type of depth buffer today. However ubiquitous, z-buffers have their drawbacks. Due to the mathematics involved, the generated z values in a z-buffer tend not to be distributed evenly across the z-buffer range (typically 0.0 to 1.0, inclusive).

Specifically, the ratio between the far and near clipping planes strongly affects how unevenly z values are distributed. Using a far-plane distance to near-plane distance ratio of 100, 90 percent of the depth buffer range is spent on the first 10 percent of the scene depth range. Typical applications for entertainment or visual simulations with exterior scenes often require far-plane/near-plane ratios of anywhere between 1,000 to 10,000. At a ratio of 1,000, 98 percent of the range is spent on the first 2 percent of the depth range, and the distribution becomes worse with higher ratios. This can cause hidden surface artifacts in distant objects, especially when using 16-bit depth buffers, the most commonly supported bit-depth.

A w-based depth buffer, on the other hand, is often more evenly distributed between the near and far clip planes than a z-buffer. The key benefit is that the ratio of distances for the far and near clipping planes is no longer an issue. This allows applications to support large maximum ranges, while still getting relatively accurate depth buffering